UNIVERSITÉ PARIS SACLAY
Master E3A – SETI

Examen Architecture T1
Novembre 2021

3h – Notes de cours autorisées pour la deuxième partie de l'examen.

Partie 2

1 Optimisation de boucles

On utilise le processeur superscalaire défini dans l'annexe 1, ou sa version scalaire.

Soit la boucle suivante écrite en assembleur, qui travaille sur des tableaux de `floats` `x[256]` et `y[256]`. Au démarrage `r1` contient l'adresse de `x[0]` et `r2` contient l'adresse de `y[0]`. `r4` contient la valeur 256. `f31` contient un nombre flottant quelconque `a`, et `f0` la valeur `0.0f`.

```
1  Boucle:
2      lf   f1, 0(r1)
3      lf   f2, 0(r2)
4      fmul f1,f1,f31
5      fadd f2,f1,f2
6      fadd f0,f0,f2
7      addi r1, r1,4
8      addi r2, r2,4
9      addi r4, r4,-1
10     bne  r4, r0, Boucle
```

1.1 Donner le programme C équivalent.

1.2 Indiquer les dépendances du programme et l'optimiser par réordonnancement en permutant des instructions assembleur.

Donner l'exécution cycle par cycle de la boucle avant et après optimisation dans la version scalaire du processeur. Quel est, en nombre de cycles, le temps d'exécution par itération de la boucle ?

1.3 Montrer également que l'on peut améliorer le temps d'exécution scalaire en changeant l'ordre des calculs et modifiant le code asssembleur. Le résultat final obtenu sera-t-il *exactement* identique ?

(On pourra au choix partir du code de la question 1.2 ou 1.3 pour le reste de l'exercice.)

1.4 On considère maintenant la version surperscalaire du processeur. Donner l'exécution cycle par cycle de la boucle optimisée en plaçant les instructions dans les différents pipelines E0, E1, FA et FM comme sur le tableau ci-dessous. Quel est, en nombre de cycles, le temps d'exécution par itération de la boucle ?

cycles	E0	E1	FA	FM
1				
2				
...				

1.5 On veut optimiser *le plus possible* le temps d'exécution et pour cela on se propose de faire un déroulage de boucle d'ordre 4.

Proposer une version en C du programme optimisé par déroulage de boucle.

Donner la version en assembleur correspondante et l'exécution cycle par cycle de la boucle en indiquant l'occupation des différents pipelines.

2 Branchements

Soit une boucle dans un programme comprenant 4 branchements 1 à 4 qui ont le comportement suivant sur 8 itérations (P pour pris et N pour non pris).

Itération	1	2	3	4	5	6	7	8
Branch 1	N	P	P	N	N	P	P	N
Branch 2	P	P	P	P	P	N	N	N
Branch 3	P	N	N	P	P	P	P	N
Branch 4	P	P	P	P	P	P	P	N

2.1 On dispose des prédicteurs suivants :

— statique toujours pris

— dynamique local 1 bit (initialisé à pris)

— dynamique local 2 bits (initialisé à fortement pris)

Déterminer pour chacun des branchements le taux de bonnes prédictions pour chacun des prédicteurs ci-dessus.

Suivant le branchement, quel est le meilleur prédicteur ? Et globalement ?

3 Renommage de registres

Nous supposons un processeur ayant 8 registres architecturaux `r0` à `r7` et 8 registres physiques additionnels nommés `p0` à `p8`.

Ce processeur contient une table d'état simplifiée qui indique pour chaque registre s'il est en cours d'utilisation (à 1 s'il va être écrit par une instruction précédente) ou non (à 0). Il contient également une table des alias de registres (RAT) qui indique pour chaque registre architectural le registre (architectural ou physique) par lequel il a été renommé.

3.1 A un certain moment, les tables d'état et la RAT ont le contenu suivant

Table d'état `r`

registre	r0	r1	r2	r3	r4	r5	r6	r7
état	1	1	1	1	1	1	1	1

Table d'état `p`

registre	p0	p1	p2	p3	p4	p5	p6	p7
état	0	0	0	0	0	0	0	0

RAT

registre	r0	r1	r2	r3	r4	r5	r6	r7
renommé	r0	r1	r2	r3	r4	r5	r6	r7

Autrement dit, tous les registres architecturaux sont utilisés, tous les registres physiques sont disponibles et aucun registre architectural n'a été renommé en un registre physique.

Donner la version après renommage du code ci-dessous en faisant apparaître les registres effectivement utilisés(par exemple, si vous pensez que le registre `r3` a été renommé en `p6` pour l'instruction 1, répondez 1 `r1 <- r2 + p6`). Donner également les tables d'état et la RAT après l'exécution de ces instructions. Nous supposerons qu'aucune instruction ne se sera achevée durant l'exécution.

```
1 r1 <- r2+r3
2 r3 <- r4+r5
3 beqz  r1
4 r1 <- r1 + r3
5 r1 <- r1 + r3
6 r3 <- r1 + r3
7 r4 <- r3 + r1
```

4 TLB et caches

Un processeur a les caractéristiques suivantes :

— adressage mémoire par octet

— les adresses virtuelles sont sur 16 bits et les adresses physiques sur 12 bits

— une page fait 32 octets

— Le TLB a 16 entrées avec une associativité de 4

4.1 Le CPU fait un accès mémoire à l'adresse virtuelle suivante : `0x024a`
En justifiant votre réponse, donner en binaire ou en hexadécimal :

— le déplacement dans la page virtuelle

— l'adresse de page virtuelle

— l'index utilisé par le TLB

— l'étiquette utilisée par le TLB

4.2 Le cache L1 de ce CPU a une capacité de 512 octets et des lignes de 4 octets. Il est à étiquette physique et index virtuel et les concepteurs ont fait en sorte qu'il ait la plus petite associativité possible.
Donner les caractéristiques de ce cache.
Pour l'adresse de la question précédente, donner la valeur du déplacement dans la ligne et de l'index.

5 Défauts de cache

On considère un cache de 128 octets, dont chaque ligne peut contenir 4 doubles. Nous considèrerons les deux cas suivants : cache à correspondance directe et cache associatif 2 voies avec remplacement utilisant une politique LRU.

Soit le programme suivant :

```
double A[32], t = 0;
for(int i = 0; i < m; i++)
  for(int j = 0; j < n; j++)
      t += A[j*m + i];
```

m, n = 32

Nous supposerons que A[0] va dans la ligne 0 del'ensemble 0 du cache et nous chercherons à déterminer le taux d'échec (nombre d'échec par accès mémoire) lors de l'exécution du programme.

5.1 Déterminer le taux d'échec pour les cas à correspondance directe et associatif 2 voies si $m = 1$.

5.2 Même question si $m = 2$.

5.3 Même question si $m = 16$.

6 Optimisation de programmes

Soit un tableau d'entiers `t`. On veut calculer indépendemment la somme des éléments positifs et négatifs du tableau `t` au moyen du code suivant :

```
int t[N], sumP=0, sumN=0;
...
for(int i=0; i<N; i++) {
  if (t[i]>=0)
    sumP += t[i] ;
  else
    sumN += t[i] ;
}
```

6.1 A l'exécution du programme, on constate suivant les caractéristiques du tableau `t` des variations du temps d'exécution très importantes pouvant aller jusqu'à un facteur 10. A votre avis, pour quelles raisons a-t-on ces variations ? Quand verra-t-on le temps d'exécution le plus faible ?

6.2 On veut réécrire le programme pour réduire le temps d'exécution et le rendre indépendant des caractéristiques du tableau `t`. Proposer pour cela une nouvelle version du programme en C.

7 Caches[1]

1 Question optionnelle et un peu plus difficile

On suppose un processeur ayant des adresses sur 8bits associé à un cache utilisant une politique de remplacement LRU.

On suppose que le cache est initialement vide et que avec la séquence d'adresses mémoire suivante, le cache a le comportement indiqué :

N°	adresse	action dans le cache
1	0	échec
2	2	succès
3	4	échec
4	0x80	échec
5	0	succès
6	0x80	succès
7	0x40	échec
8	4	succès
9	0	échec
10	0x20	échec
11	0x40	succès

7.1 Montrer que l'on peut déduire de cette séquence que le cache a nécessairement des lignes de 4 octets, une associativité 2 voies et que sa capacité totale est de 128 octets.

ANNEXE 1

Soit un processeur superscalaire à ordonnancement statique qui a les caractéristiques suivantes :

- les instructions sont de longueur fixe (32 bits)
- Il a 32 registres entiers (dont R0=0) de 32 bits et 32 registres flottants (de F0 à F31) de 32 bits.
- Il peut lire et exécuter 4 instructions par cycle.
- L'unité entière contient deux pipelines d'exécution entière sur 32 bits, soit deux additionneurs, deux décaleurs. Tous les court-circuits possibles sont implantés.
- L'unité flottante contient un pipeline flottant pour l'addition et un pipeline flottant pour la multiplication.
- L'unité Load/Store peut exécuter jusqu'à deux chargements par cycle, mais ne peut effectuer qu'un *load* et un `store` simultanément. Elle ne peut effectuer qu'un seul `store` par cycle.
- Le processeur dispose d'un mécanisme de prédiction de branchement qui permet de *brancher* en 1 cycle si la prédiction est correcte. Les sauts et branchements ne sont pas retardés.

La Table 1 donne les instructions disponibles et le pipeline qu'elles utilisent : E0 et E1 sont les deux pipelines entiers, FA est le pipeline flottant de l'addition et FM le pipeline flottant de la multiplication. Les instructions peuvent être exécutées simultanément si elles utilisent chacune un pipeline séparé. L'addition et la multiplication flottante sont pipelinées. La division flottante n'est pas pipelinée (une division ne peut commencer que lorsque la division précédente est terminée). L'ordonnancement est statique.

JEU D'INSTRUCTIONS (extrait)

Code	Instruction	Latence	Pipeline	Signification
LF	LF Fi, dép.(Ra)	2	E0 ou E1	Fi ← M (Ra + dépl.16 bits signé)
SF	SF Fi, dép.(Ra)	0	E0	Fi → M (Ra + dépl.16 bits signé)
ADD	ADD Rd,Ra, Rb	1	E0 ou E1	Rd ← Ra + Rb
ADDI	ADDI Rd, Ra, IMM	1	E0 ou E1	Rd ← Ra + IMM (16 bits signé)
SUB	SUB Rd,Ra, Rb	1	E0 ou E1	Rd ← Ra - Rb
FADD	FADD Fd, Fa, Fb	3	FA	Fd ← Fa + Fb
FMUL	FMUL Fd, Fa, Fb	3	FM	Fd ← Fa × Fb
BEQ	BEQ Ri, dépl	1	E1	si Ri=0 alors CP ← NCP + depl
BNE	BNE Ri, dépl	1	E1	si Ri≠ 0 alors CP ← NCP + depl